



UNIVERSITÀ DI PISA

Department of Computer Science

Course: Distributed Data Analysis and Mining

Distributed Data Analysis of 6.25 Million Chess Games using PySpark

Student:

Giorgio Gobbin

Matricola: 683071

January 29, 2026

Abstract

In this project, I analyze a dataset of 6.25 million chess games played on Lichess using **PySpark**. My goal is to uncover patterns in player behavior, predict game outcomes, and analyze opening strategies across different skill levels. By using Random Forest classifiers (achieving $\sim 63\%$ accuracy) and K-Means clustering ($k = 4$), I identified distinct player profiles, including a specific cluster of "Marathon" games. Furthermore, through Pattern Mining, I discovered specific opening traps that significantly alter win probabilities.

Contents

1	Introduction	3
2	Dataset and Computing Environment	3
2.1	The Lichess Dataset	3
2.2	Understanding the ELO System	3
2.3	Variable Description	3
2.4	Computing Environment	4
3	Data Understanding and Quality Assessment	4
3.1	Data Quality Check	5
3.2	Statistical Distribution	5
3.3	Correlation Analysis	5
4	Data Processing Pipeline	6
4.1	Cleaning and Feature Engineering	6
4.2	Advanced Selection with Window Functions	6
5	Supervised Learning: Predicting the Winner	7
5.1	Evaluation	7
5.2	Feature Importance	7
6	Unsupervised Learning: Player Segmentation	8
6.1	Optimizing k (Elbow Method)	8
6.2	Cluster Interpretation	9
7	Pattern Mining: Association Rules	9
7.1	The "Van't Kruijs" Trap	9
8	Regression Analysis: Predicting Game Length	10
9	Conclusions	11

1 Introduction

This project processes a large-scale dataset to answer the following Research Questions (RQs):

- **RQ1 (Supervised):** Can I predict the winner based on ELO ratings and opening strategies?
- **RQ2 (Unsupervised):** Can players (and games) be segmented into distinct "style profiles"?
- **RQ3 (Mining):** Are there hidden patterns or rules connecting specific openings to game outcomes?

2 Dataset and Computing Environment

2.1 The Lichess Dataset

For this analysis, I selected the *Lichess Standard Games* dataset, containing games played on the *lichess.org* server during **July 2016**. This dataset represents a significant Big Data challenge with over **4 GB** of information.

The dataset contains **6,256,184 chess games**. It combines structured metadata (ratings, results) with unstructured move sequences.

2.2 Understanding the ELO System

A crucial component of this dataset is the **ELO rating system**, a method used to calculate the relative skill levels of players in zero-sum games. The core principle is that a player's rating represents their probability of winning. After every game, the winner takes points from the loser:

- If a high-rated player beats a low-rated one, their rating increases only slightly because the result was expected.
- If a low-rated player beats a high-rated one, their rating increases significantly.

In this dataset, ratings range from ~ 700 (Beginner) to over 3000 (Super Grandmaster), with an average around 1500-1700.

2.3 Variable Description

The raw dataset consists of the following key features:

- **Game Metadata:**
 - **Event:** The type of game (e.g., Blitz, Classical tournament).
 - **White / Black:** Anonymized User IDs for both players.
 - **Result:** The outcome (1-0 for White win, 0-1 for Black win, 1/2-1/2 for Draw).
 - **UTCDate / UTCTime:** Timestamp of the match.
- **Player Metrics:**

- **WhiteElo / BlackElo**: The ELO rating of the players at the start of the game.
- **WhiteRatingDiff / BlackRatingDiff**: The rating points gained or lost after the match based on the ELO formula.

- **Game Specifics:**

- **ECO / Opening**: The opening strategy encoded via the Encyclopedia of Chess Openings (ECO) system, along with the full opening name.
- **TimeControl**: Time setting in seconds. The format (e.g., "300+0") indicates the total time per player plus the increment in seconds added before every move.
- **Termination**: The reason for the game ending (e.g., Normal, Time forfeit, Abandoned).
- **AN**: The full move sequence in Algebraic Notation (Movetext format). This column is the largest and most complex, containing the entire history of the game.

```
root
|-- Event: string (nullable = true)
|-- White: string (nullable = true)
|-- Black: string (nullable = true)
|-- Result: string (nullable = true)
|-- UTCDate: string (nullable = true)
|-- UTCTime: string (nullable = true)
|-- WhiteElo: string (nullable = true)
|-- BlackElo: string (nullable = true)
|-- WhiteRatingDiff: string (nullable = true)
|-- BlackRatingDiff: string (nullable = true)
|-- ECO: string (nullable = true)
|-- Opening: string (nullable = true)
|-- TimeControl: string (nullable = true)
|-- Termination: string (nullable = true)
|-- AN: string (nullable = true)
```

Figure 1: PySpark DataFrame Schema showing the raw data structure.

2.4 Computing Environment

I utilized **Kaggle Notebooks**, leveraging the **NVIDIA T4 GPU x2** accelerator to speed up operations where applicable. The analysis was implemented using **PySpark 3.x**.

3 Data Understanding and Quality Assessment

Before modeling, I performed a comprehensive Data Quality Assessment to validate the dataset's integrity.

3.1 Data Quality Check

The dataset demonstrates exceptionally high quality:

- **Duplicate Analysis:** Out of 6,256,184 total rows, I identified only **1 duplicate row**.
- **Missing Values:** Core columns (AN, Result, ECO) have zero missing values. Negligible missing data was found only in rating difference columns ($< 0.07\%$).

I delete both the Duplicate and the missing values.

3.2 Statistical Distribution

Descriptive statistics reveal the nature of the player base:

- **ELO Rating:** The average rating is ~ 1741 (StdDev ~ 266), ranging from 737 to 3110. The distribution follows a near-perfect Gaussian curve (Figure 2).
- **The "1500 Spike":** As clearly visible in Figure 2, there is a sharp spike exactly at the 1500 mark. This is not a statistical anomaly, but an artifact of the Lichess rating system: 1500 is the default entry point assigned to new accounts (Glicko-2 provisional rating). These users have likely played very few games, retaining their starting score.
- **Game Termination:** As shown in Figure 3, the majority of games (67.6%) end normally (Checkmate or Resignation), while a significant portion (32.1%) end by "Time forfeit".

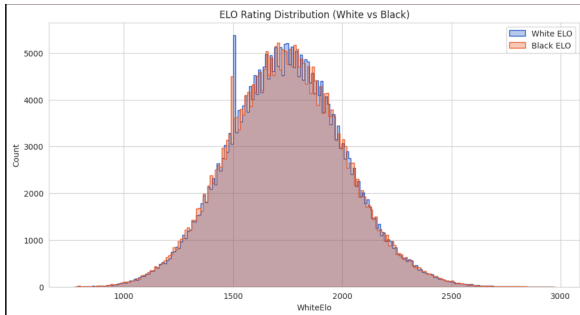


Figure 2: ELO Rating Distribution. Note the artificial spike at 1500 (default provisional rating).

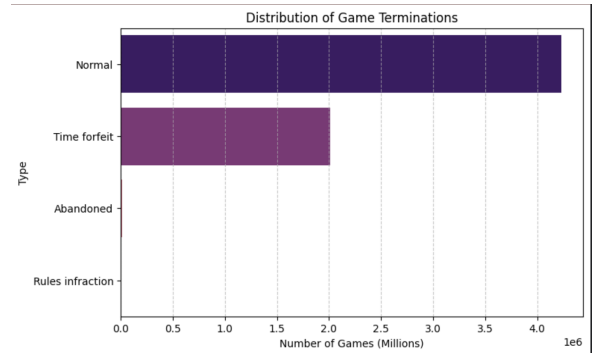


Figure 3: Game Termination Types.

3.3 Correlation Analysis

I computed the correlation matrix for all numerical features (Figure 4).

- **Matchmaking Efficiency (0.71):** The strong positive correlation between `WhiteElo` and `BlackElo` confirms effective matchmaking.
- **Skill Gap vs. Duration (-0.016):** Surprisingly, the correlation between `EloDiff` and `GameLength` is near zero, implying that skill gaps do not necessarily lead to shorter games.

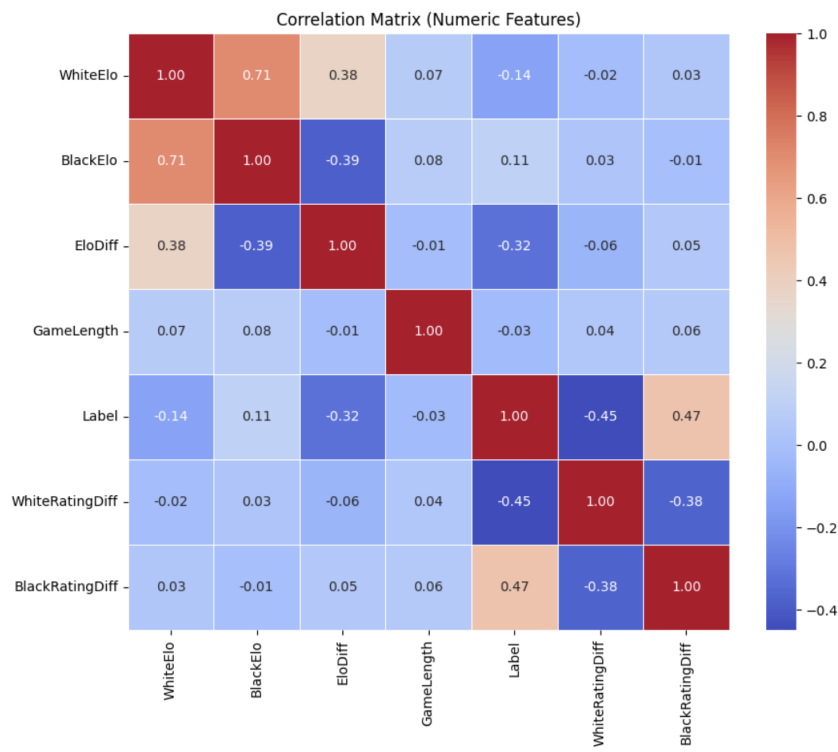


Figure 4: Correlation Matrix of numeric features.

4 Data Processing Pipeline

Preprocessing is crucial for the next parts and because of that I performed the following steps:

4.1 Cleaning and Feature Engineering

I filtered out abandoned games and converted the **Result** column into a numerical label (0=Draw, 1=White, 2=Black). I inserted new features such as **EloDiff** (White ELO - Black ELO) and **GameLength** (derived from the lenght of the records in the AN's column).

4.2 Advanced Selection with Window Functions

To identify the "reference games" for each opening strategy without using expensive self-joins, I utilized PySpark **Window Functions**. Specifically, I partitioned the data by ECO code and ranked games by ELO rating to extract the single best game for each opening.

```

1 # Window Function: Find highest rated game per Opening
2 spec = Window.partitionBy("ECO").orderBy(desc("WhiteElo"))
3
4 df_ranked = df.withColumn("Rank", row_number().over(spec))
5 top_games = df_ranked.filter(col("Rank") == 1)

```

Listing 1: Using Window Functions to find top games

ECO	Opening	White ELO	Result
A00	Van Geet Opening	3058	1-0
A01	Nimzo-Larsen Attack	3106	1-0
A03	Bird Opening: Dutch Variation	2799	1-0
A09	Reti Opening	2801	1-0
A11	English Opening: Caro-Kann Defensive System	2683	1-0

Table 1: Top-rated games for selected openings identified via Window Functions.

5 Supervised Learning: Predicting the Winner

I trained a **Random Forest Classifier** to predict the game outcome. During the training phase, I encountered a high-cardinality issue with the `ECO` variable. I resolved this by increasing the `maxBins` hyperparameter to 1000, allowing the model to handle all opening variations correctly.

5.1 Evaluation

My model achieved an accuracy of **63%**, significantly outperforming random guessing (33% for 3 classes).

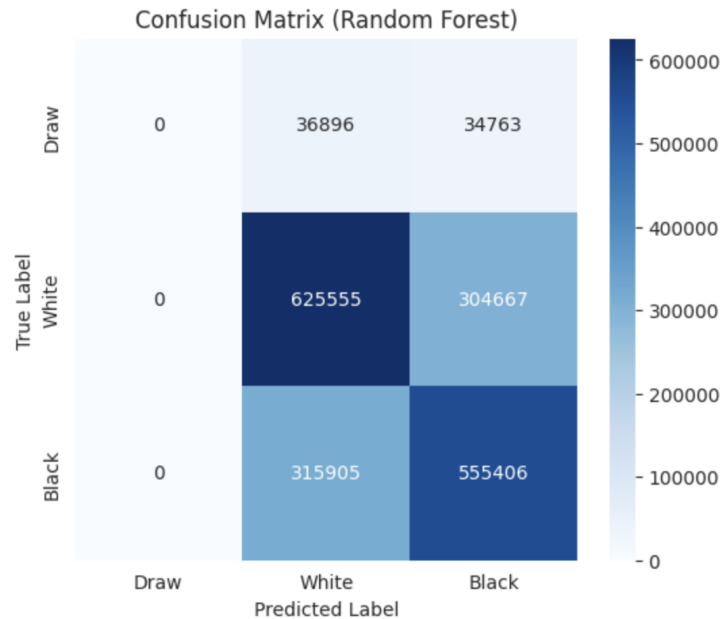


Figure 5: Confusion Matrix. The model favors decisive results over Draws.

5.2 Feature Importance

As shown in Figure 6, `EloDiff` is the dominant predictor by a significant margin, with an importance score near 0.8. This confirms that the skill gap between players is the most reliable factor for predicting the game's outcome.

Interestingly, while `WhiteElo` and `BlackElo` contribute moderately to the model, `GameLength` shows a very low importance score. This quantitative finding aligns with

my later regression analysis, which showed a near-zero R^2 score, confirming that the duration of a match is not a meaningful indicator of who will emerge victorious or of the initial skill imbalance.

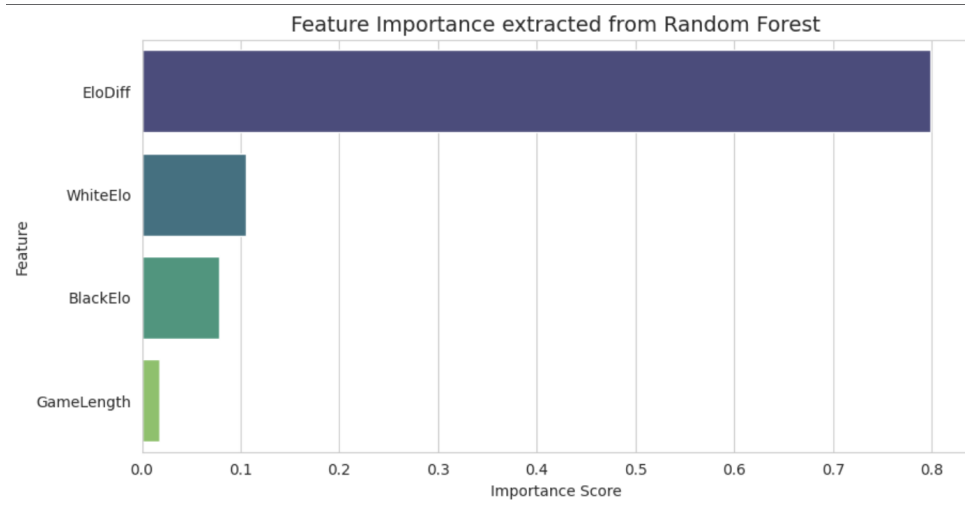


Figure 6: Feature Importance extracted from Random Forest model.

6 Unsupervised Learning: Player Segmentation

I applied **K-Means Clustering** to segment games based on `AverageElo`, `GameLength`, and `EloGap`.

6.1 Optimizing k (Elbow Method)

To determine the optimal number of clusters, I utilized the **Elbow Method**, calculating the Within-Cluster Sum of Squared Errors (WCSS) for $k = 2$ to $k = 8$.

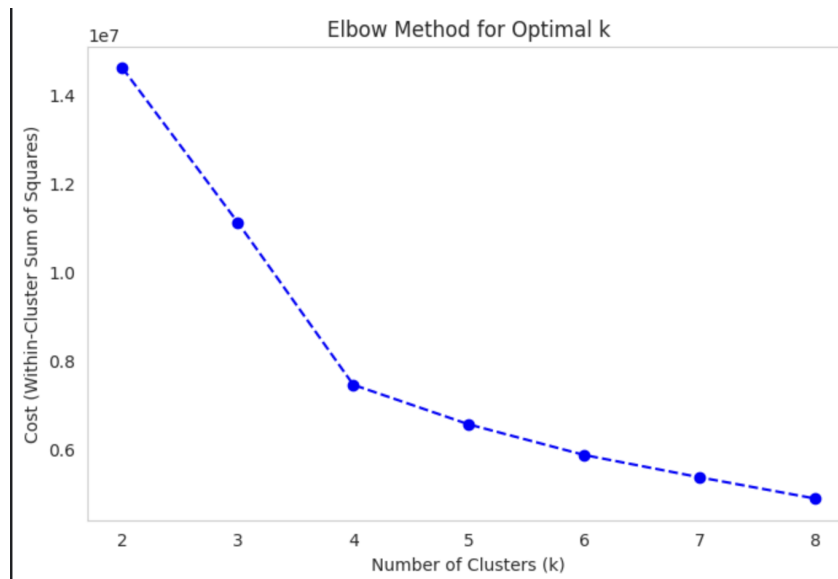


Figure 7: Elbow Method showing the optimal k at 4.

As shown in Figure 7, the curve flattens significantly after $k=4$, justifying my choice of 4 clusters for the segmentation.

6.2 Cluster Interpretation

The algorithm identified four distinct player profiles. Looking at the feature distribution in Figure 8, I characterized them as follows:

- **Cluster 0 (Masters):** The highest skill group (Avg ELO ~ 1952). These players play standard length games with very low Elo Gaps, indicating highly competitive matchmaking at the top level.
- **Cluster 1 (Beginners):** The lowest skill group (Avg ELO ~ 1545). Characterized by the shortest game duration (~ 407 chars), likely due to early resignations or tactical oversights.
- **Cluster 2 (Mismatches):** Defined almost exclusively by a massive EloGap (~ 407 points vs the average of 100). These are games where a very strong player faced a much weaker one.
- **Cluster 3 (Marathons):** While having an average ELO (~ 1745), these games are exceptionally long (~ 1897 chars), nearly 4x the average. These represent distinctively complex strategic battles or heavy maneuvering endgames.

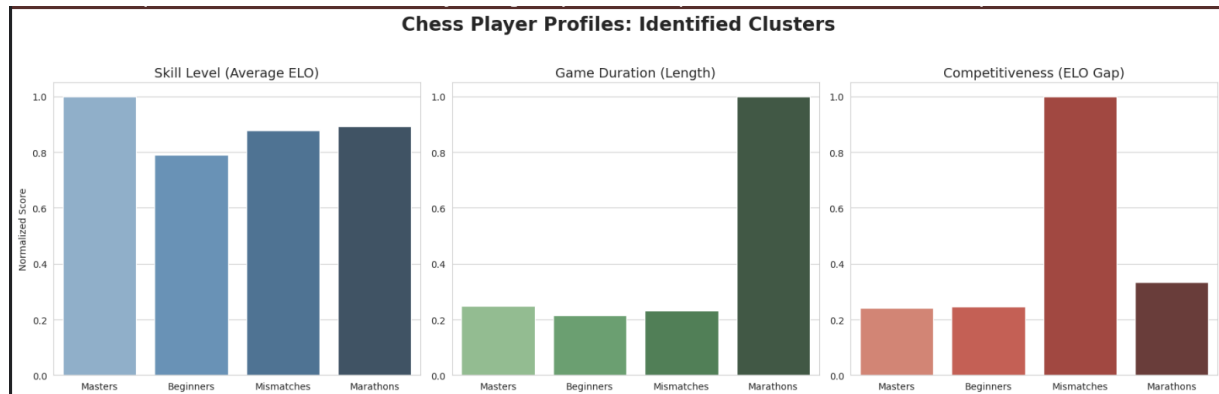


Figure 8: Feature comparison across the 4 identified clusters (Masters, Beginners, Mismatches, Marathons).

7 Pattern Mining: Association Rules

Beyond simple prediction, I used the **FPGrowth** algorithm to discover frequent patterns and association rules between Opening strategies and Game Results.

7.1 The "Van't Kruijs" Trap

The analysis uncovered a highly significant association rule with a confidence of **53.1%**:

$$\{Van'tKruijsOpening\} \Rightarrow \{BlackWin\}$$

This rule flags the **Van't Kruijs Opening** (1. e3) as a statistical weakness for White. As shown in Figure 9, unlike standard moves (e.g., 1. e4) that aggressively fight for the center, 1. e3 is passive. It restricts the Queen's Bishop and voluntarily surrenders the "first-move advantage," effectively handing the initiative to Black immediately. The data validates this theoretical flaw: this passive setup correlates with a sharp increase in the opponent's win rate.



Figure 9: The Van't Kruijs Opening (1. e3). A passive move that fails to control the center, giving the initiative to Black.

Antecedent (Opening)	Consequent (Result)	Confidence
Van't Kruijs Opening	Black Win	0.531
Scandinavian Defense	Black Win	0.485

Table 2: Top Association Rules extracted by FPGrowth.

8 Regression Analysis: Predicting Game Length

I attempted to predict the duration of the game (**GameLength**) using **Linear Regression** based on the skill difference (**EloDiff**).

- **Hypothesis:** Higher skill difference leads to shorter games (quick mate).
- **Result (R^2):** 0.0003
- **Coefficient:** -0.0405

The R^2 score is effectively zero. This negative result is scientifically important: it disproves the hypothesis. A large skill gap does **not** statistically guarantee a shorter game on Lichess; Maybe the stronger players can make fun of the weaker ones by continuing to play instead of an easy checkmate.

9 Conclusions

This project demonstrated that while ELO rating is the primary predictor of chess outcomes, the style of play evolves drastically with skill level. Key findings include:

1. **Predictability:** Random Forests perform with $\sim 63\%$ accuracy, driven heavily by the `EloDiff` feature.
2. **Clustering:** K-Means successfully identified a specific cluster of "Marathon Games" (very long matches), proving that game length is a fundamental discriminator in chess data.
3. **Opening Traps:** Association Rule Mining revealed that passive openings like the *Van't Kruijs* statistically flip the odds in favor of Black (53% win rate).

Using PySpark allowed me to efficiently process 6.25 million records to derive these insights.